Discrete Bit Loading for DMT: Linear Scan versus Priority Queue

EEP55C36 Computational Methods and Algorithms — Final Project, Track D

Inderjeet Singh Sukhmani | Trinity College Dublin

November 2026

Abstract

I reproduce the greedy marginal-energy core of Campello's discrete bit-loading framework for discrete multitone (DMT) transmission and change one step. The tone that receives the next bit is chosen from a binary min-heap instead of a full linear scan. On 30 paired synthetic test channels per setting, the modified method returned the *identical* allocation in all 1186 method runs. On my laptop it was 13–14× faster for 64–1024 tones and 16.7× faster for 4096. The predicted growth of the speed-up with N appeared only weakly, because NumPy vectorisation hides the scan's $O(N)$ cost. With a tiny energy budget the advantage disappeared (1.16×, 95% CI $[0.96, 1.40]$). A calibration stress test also shows that planning on estimated channel gains overspends the true energy budget, whichever method is used.

1 Problem and paper

Discrete multitone (DMT) modulation, used in DSL, splits a wired channel into N narrow subchannels called *tones*. Tones with a strong channel and little noise can carry more bits. *Bit loading* decides how many bits, and therefore how much energy, each tone gets, so that the total rate is maximised within an energy budget. Campello [1] gives a practical framework for this integer problem. **Reproduction scope:** only the *discrete marginal-energy (greedy) allocation core*, starting from zero bits with a per-tone cap. The paper's initialisation and acceleration steps are not reproduced. All data are **synthetic** (Sec. 4), and no physical bit-error performance is claimed.

2 Mathematics and implementation

For tone $i \in \{0, \dots, N-1\}$, $y_i = h_i x_i + n_i$, where h_i is the complex channel gain and n_i is zero-mean noise with variance $\sigma_i^2 > 0$. With transmitted energy $e_i = \mathbb{E}|x_i|^2$ (normalised units), the received SNR is $g_i e_i$, where $g_i = |h_i|^2/\sigma_i^2$ is the channel-to-noise ratio per unit energy. Under the *gap approximation* with gap $\Gamma \geq 1$ (the SNR penalty of a practical modulation and code relative to capacity), carrying b_i bits needs

$$e_i(b_i) = \frac{\Gamma}{g_i}\left(2^{b_i} - 1\right), \qquad \max_{\mathbf{b}} \; B = \sum_i b_i \quad \text{s.t.} \quad \sum_i e_i(b_i) \leq E_{\max}, \quad b_i \in \{0, 1, \dots, b_{\max}\}. \tag{1}$$

The *optimisation objective* B (total bits) is not the same as engineering task quality: a predicted rate is reliable only if g and Γ are correct. The marginal cost of one more bit on tone i is

$$\Delta_i(b_i) = e_i(b_i{+}1) - e_i(b_i) = \frac{\Gamma}{g_i}\, 2^{b_i}, \tag{2}$$

which *doubles* after every bit. **Greedy rule.** Start from $\mathbf{b} = \mathbf{0}$ and repeatedly buy the cheapest increment among tones with $b_i < b_{\max}$, breaking ties by the smallest index. Stop when that increment costs more than the remaining energy. **Why it is optimal for this model:** each tone's costs strictly increase, and bit k cannot be bought without bits $1, \dots, k{-}1$. So for every B the B globally cheapest increments form the cheapest feasible set, and once the next one is unaffordable, no allocation with $B{+}1$ bits fits. **Assumption and failure condition:** the argument needs independent tones with separable costs. It fails with coupled tones (e.g. crosstalk between DSL lines), unequal bit steps or a different objective.

Verification (`tests/`, 348 checks, `python -m pytest`). *Hand-computed cases:* $g = [1, 1]$, $\Gamma = 1$ gives $[1, 1]$ at $E_{\max} = 3$ and $[2, 1]$ at $E_{\max} = 4$. The course guide quotes $[2, 1]$ "using energy 3" at $E_{\max} = 3$, but $e(2) + e(1) = 4 > 3$, so that allocation is infeasible under the stated model, and exhaustive search agrees with my value. *Limits:* zero budget, a budget below the cheapest bit, $b_{\max} = 0$, saturation at the cap and empty input all behave correctly, and invalid inputs raise `ValueError`. *Optimality:* on 40 random tiny cases, B equals the exhaustive-search optimum, with the minimum energy for that B. *Properties:* energy used $\leq E_{\max}$ and matches the closed form; outputs are finite, shape $(N,)$ and integer; B never decreases with the budget; a stronger tone never gets fewer bits; inputs are never mutated. These checks establish *code correctness* for the model. They say nothing about whether the model describes a real line (Sec. 5).

Baseline `src/baseline.py` — $\Theta(N)$ per increment

```
cost = gap / g # Delta_i(0), eq. (2)
while n_tones > 0:
    eligible = np.where(bits < bmax, cost, np.inf)
    i = int(np.argmin(eligible)) # first index on ties
    step = eligible[i]
    if step > remaining: # also stops if all capped
        break
    remaining -= step; bits[i] += 1
    cost[i] *= 2.0 # Delta doubles
```

Modified `src/modified.py` — $O(\log N)$ per increment

```
heap = [(c, i) for i, c in enumerate(gap / g)]
heapq.heapify(heap) # O(N) setup
while heap:
    cost, i = heap[0] # (cost, index) order
    if cost > remaining:
        break
    remaining -= cost; bits[i] += 1
    if bits[i] < bmax: heapq.heapreplace(heap, (2*cost, i))
    else: heapq.heappop(heap)
```

Figure 1: The only algorithmic difference is how the cheapest increment is found (code simplified from the repository; input validation and bookkeeping omitted).

3 The change: priority-queue selection

What changes. Only the *selection step* (Fig. 1). The next-bit costs (2) are kept in a binary min-heap of (Δ_i, i) pairs (Python `heapq`). The cheapest increment is read from the top in $O(1)$. After a purchase, that entry is replaced by $(2\Delta_i, i)$ and sifted down in $O(\log N)$, and a tone reaching $b_{\max}$ is popped. Python compares tuples element by element, so equal costs are ordered by index, which reproduces `argmin`'s first-index tie rule. The energy model, the stopping rule and the order of floating-point operations are unchanged, and doubling is exact in binary. **The output should therefore be bit-for-bit identical, and only the cost changes:** $O(N)$ setup plus $O(B \log N)$, against $\Theta(NB)$. Priority queues for repeated minimum selection are standard [2], and the course guide [3] lists this change for the track. The implementation is my own. Setup (`heap_setup`) and allocation (`heap_allocate`) are separate functions so that their costs can be timed separately.

Hypothesis: identical allocations, with a speed advantage that grows with N. **Predicted to help** when there are many tones and many increments. **Predicted to hurt** with tiny budgets, where the $O(N)$ heap build is not repaid. A further risk is that the baseline's `argmin` is vectorised C code, while the heap runs in interpreted Python. **Ablation:** `selector="list-scan"` keeps every other decision of the modified code (Python lists, same loop, same arithmetic) and switches off only the heap, using a linear Python scan instead. This separates the data-structure effect from the NumPy-versus-Python effect.

4 Experiments

Data. The synthetic 8-tap multipath channel follows the course guide: tap powers $\propto e^{-\ell/2}$ normalised to 1, $\sigma^2 = 0.1$, and $h = \mathrm{FFT}_N(\text{taps})$ (`src/data.py`). There are two information conditions, given identically to both methods: *true* g, and *calibrated* $\hat{g}$ from 20 averaged unit pilots and 20 independent noise-only observations. The model settings are $\Gamma = 9.8\,\mathrm{dB}$ (uncoded QAM, symbol-error rate $\approx 10^{-7}$) and $b_{\max} = 15$. **Separation of data.** Seeds 0–19 (development) were used for debugging and tests. Seeds 100–119 (validation) were used only to choose the energy per tone ρ in $E_{\max} = \rho N$, by the rule "mean bits/tone closest to 5" (mid-range, so neither budget-starved nor cap-saturated). This gave $\rho = 40$ (4.80 bits/tone). Seeds 1000–1029 (test) are used only for reported results. No supervised training is needed, and each case is an independent channel realisation. **Protocol.** The main experiment uses $N \in \{64, 256, 1024\} \times \{\text{true, calibrated}\} \times 30$ paired cases. A scaling run covers $N = 16$ to 4096. The stress cases are (i) a budget sweep $\rho \in [0.01, 40]$ at $N = 1024$; (ii) $\hat{g}$ from 4/20/100 pilots, with each allocation's energy then evaluated on the *true* g; and (iii) a perfectly flat channel, where every selection is a tie. **Metrics.** The *primary metric*, fixed in advance, is the paired speed-up $t_{\text{scan}}/t_{\text{heap}}$ per case (larger is better), summarised as a geometric mean with a 95% t-interval on the log ratios across independent channels. The secondary metrics are exact allocation agreement, B, energy, and true-budget overspend. **Timing.** Each measurement uses `time.perf_counter` with the garbage collector off, 1 warm-up run, and the median of 5 repeats. Heap setup is included in the heap time and also reported separately. All reported results were run on my laptop (AMD x86-64 CPU, Family 23 Model 160, 8 logical cores; Windows 11 build 26300; Python 3.14.3; NumPy 2.5.3; matplotlib 3.11.2), as recorded in `results/final/environment.json`. An earlier run of the same code on a 2-vCPU Linux cloud VM gave identical allocations and smaller speed-ups (about 7×).

5 Results and interpretation

Correctness of the change. The allocations were *identical* (bits, energy and remaining budget compared exactly) in all 1186 method runs, including the flat-channel tie stress (Appendix, Fig. 3). **Speed.** The heap is 13–14× faster at $N = 64$–1024, with tight CIs, and 16.7× [16.0, 17.5] at $N = 4096$ (Table 1, Fig. 2). **The hypothesis is only partly supported.** From $N = 64$ to 1024 the speed-up grows only from 13.3× to 14.0×. The NumPy scan costs about 6 µs per increment for every N from 16 to 256, and 7.4 µs at $N = 1024$, because per-call overhead dominates the vectorised `argmin` (Fig. 4a). So the measured baseline is not yet "$\Theta(N)$ per step" at these sizes, and the growth predicted by the asymptotic analysis appears only at $N = 4096$ (15.5 µs per increment). The heap stays at 0.4–0.9 µs. **The ablation** isolates the data structure. With identical Python code, heap versus linear scan gives 7.2×, 24.5× and 91.9× for

Table 1: Main comparison, $\rho = 40$, 30 paired test cases per row. Times are medians and include setup; intervals are 95% CIs of the geometric mean.

N	mean B	scan (ms)	heap (ms)	speed-up, true g	speed-up, calibrated $\hat{g}$	ablation: list-scan/heap
64	302	2.02	0.147	13.3 [12.4, 14.3]	14.0 [13.0, 14.9]	7.2 [6.7, 7.7]
256	1208	8.00	0.572	13.7 [13.1, 14.4]	14.4 [13.9, 15.0]	24.5 [23.2, 25.9]
1024	4834	35.88	2.550	14.0 [13.6, 14.4]	14.0 [13.5, 14.6]	91.9 [88.3, 95.5]

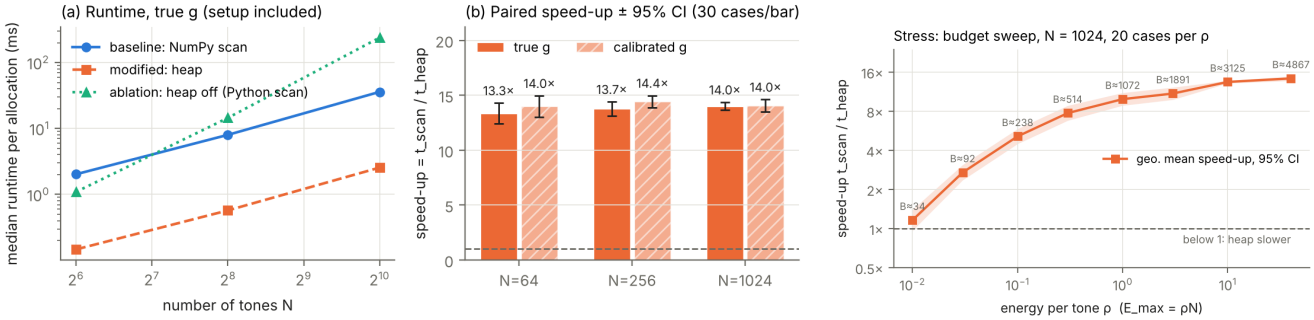


Figure 2: Left: overall comparison (runtime, and paired speed-up with 95% CI). Right: stress/failure case. As the budget shrinks, the 14× advantage disappears ($\rho = 0.01$: 1.16×, CI $[0.96, 1.40]$, about 34 bits in total). Both generated by `run_experiments.py` from `results/final/*.csv`.

$N = 64, 256, 1024$, the textbook $O(N)$-versus-$O(\log N)$ separation. Most of this gap is hidden by vectorisation in the baseline, not removed.

Table 2: Stress cases (true g unless stated). Left: budget sweep at $N = 1024$, 20 cases per ρ. Right: allocation planned on $\hat{g}$, energy then measured with the true g ($N = 256$, $\rho = 40$, 30 cases); identical for both methods.

ρ	mean B	speed-up [95% CI]
0.01	34	1.16 [0.96, 1.40]
0.03	92	2.70 [2.36, 3.09]
0.1	238	5.12 [4.48, 5.84]
1	1072	9.89 [8.84, 11.06]
40	4867	14.27 [13.70, 14.87]

pilots	mean overspend [95% CI]	max	cases over budget
4	+64.9% [51.1, 78.7]	+161.8%	30/30
20	+8.9% [7.4, 10.4]	+18.1%	30/30
100	+1.5% [1.2, 1.8]	+3.5%	29/30

Failure case. At $\rho = 0.01$ the heap build (0.26 ms, about 77% of the heap's total time) is barely repaid by about 34 increments. The speed-up falls from 14× to 1.16× with a CI that includes 1, so there is no reliable gain. On the cloud VM the same case was 0.87× (heap slower), so the sign of the effect at tiny budgets depends on the machine. By $\rho = 0.03$ the heap is clearly faster again. A practical implementation could choose the method from the expected B/N. **Model validity, not code correctness.** When bits are planned on $\hat{g}$, the true energy exceeds $E_{\max}$ in almost every case, by +8.9% on average with 20 pilots and +65% with 4 (Table 2, Fig. 4b). Noise inflates $|\hat{h}|^2$ on average, and the greedy rule preferentially loads the tones whose g was over-estimated. Both methods are affected identically, because the change cannot fix bad inputs. **Limitations.** The data are synthetic, the gap model is approximate, tones are independent, timing comes from a single laptop (background load adds noise), and only the greedy core is reproduced. The heap's advantage depends on Python and NumPy implementation details, not only on asymptotics.

6 Reproduction

Repository: `https://github.com/ijsukhmani13-star/eep55c36-bit-loading`, submitted tag `v1.0-submission` (the full commit hash is given in the submission form). The results were produced at commit `d5419a1`, as recorded in `results/final/environment.json`. The figures were later redrawn from the same CSVs after an axis-range fix in `src/figures.py`; no numbers changed. Terminal screenshots of the run are in `docs/screenshots/`. Commands (from the repository root): `python -m pip install -r requirements.txt`; `python -m pytest` (348 checks, about 3 s); `python run_experiments.py -config configs/smoke.json` (about 10 s); `python run_experiments.py -config configs/final.json` (about 3–4 min). `-figures-only` redraws figures from the saved CSVs. All tables and figures in this report come from `results/final/summary.csv`, `raw_runs.csv` and `fig*.png`.

Contribution and assistance statement. This is an individual project. The code, tests, experiments and this report were developed with substantial help from a generative-AI assistant (Claude, Anthropic), used under the module's AI policy. I chose the track, ran all experiments on my own laptop, reviewed the code and results, and can explain every assessed function and result. No external code or datasets were used. The paper is cited but not redistributed.

References

[1] J. Campello, "Practical bit loading for DMT," in *Proc. IEEE ICC*, vol. 2, 1999, pp. 801–805. doi:10.1109/ICC.1999.765384.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, ch. 6 (heaps and priority queues).

[3] D. Ramsook, "Final-project mathematical guide, Track D," EEP55C36 course material, TCD, 2026.

Appendix: supporting figures (generated by `run_experiments.py`)

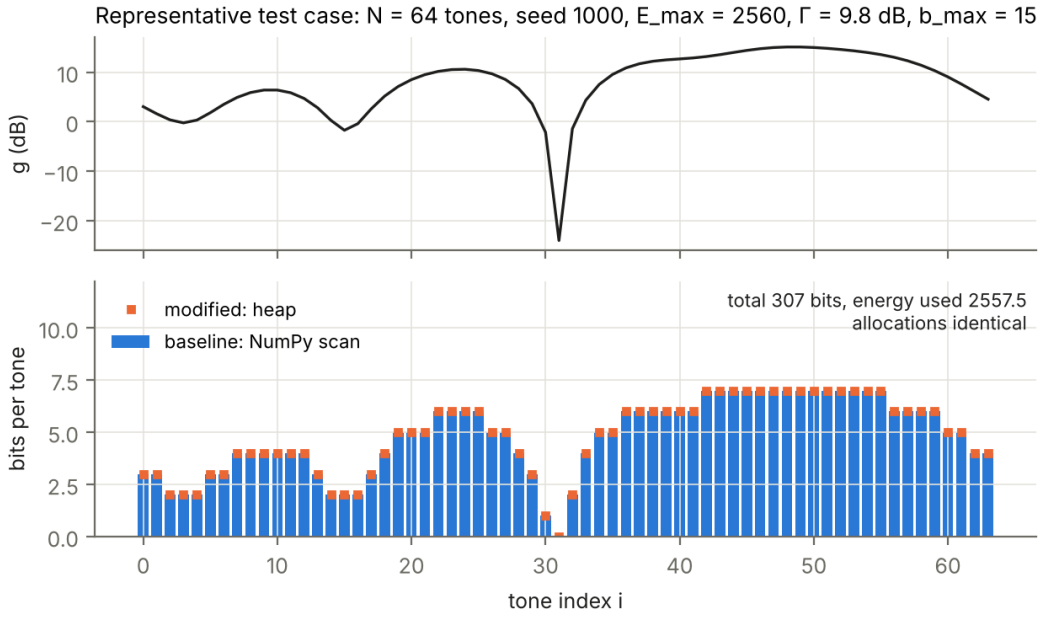
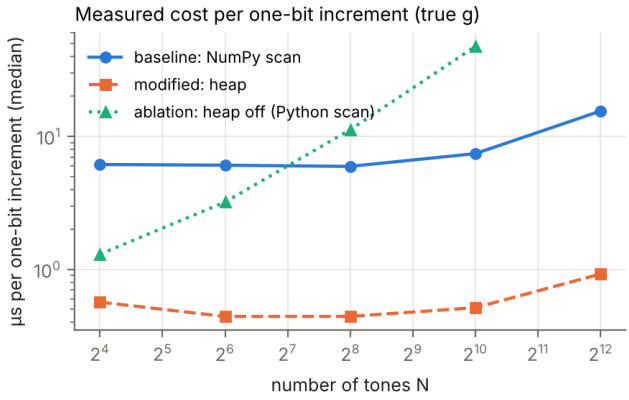
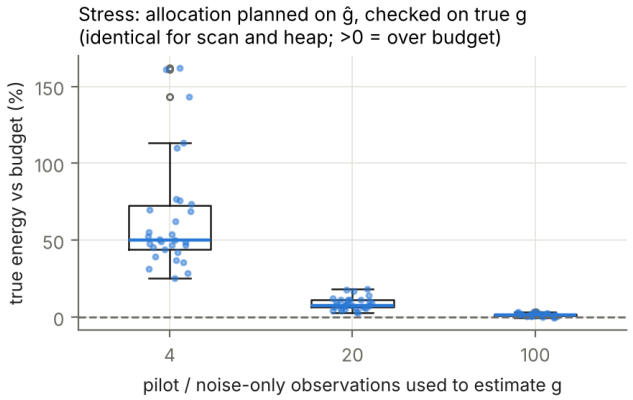


Figure 3: Representative test case ($N = 64$, seed 1000, true g). Top: channel-to-noise ratio per tone, including a deep fade at tone 31. Bottom: bits per tone from both methods, which are identical (307 bits, energy 2557.5 of 2560). Stronger tones get more bits, and the faded tone gets none.



(a) Median time per one-bit increment versus N (10 test cases per N).



(b) True-energy overspend when planning on $\hat{g}$ (30 cases per pilot count).

Figure 4: Complexity check and calibration stress test.